

Plectis: Runnable Public Components from a Private AI-Built System

What readers can run, and what those runs establish

Will Cook

July 2026

Abstract

Plectis is a public software repository containing 88 small components in 7 groups. Each component names an operation, a fixed sample input, a command that exercises it, stored output records, and a limit on what the result establishes. I developed the larger private system over about ten months by directing AI coding agents. Some Plectis components contain source bodies copied from that private repository; others are public refactors, local computations, external-tool checks, or validators. The public repository lets a reader test claims about these published files and their outputs. It does not by itself establish the history, representativeness, or general behaviour of the private system. This note defines the object in ordinary terms, shows one component from input to result, reports a clean-clone run at a named commit, and states the limits of the evidence.

1 What Plectis is

Plectis is stored at github.com/wcook04/plectis. A *repository* is an organised project folder, usually kept with its revision history. To *clone* a repository is to copy that folder and history to another computer. A *component* in Plectis is one registered, testable part of the project. Its *fixture* is a fixed sample input, and its *receipt* is a saved, machine-readable record of a reference run. These terms describe files and commands; they do not grade the importance of the result. A *validator* is the program that applies a component's pass and refusal rules. A *commit* is a named, saved version of the repository.

The argument of this paper does not require programming knowledge. Reproducing the runs does require a terminal, Git, Python 3.11 or later, `pip`, and basic command-line use. A terminal is the text window in which the commands below are entered. The final full stop in a command such as `plectis tour .` means "the current folder."

I am 22 and study economics. Over about ten months I built the private system alone in the sense relevant here: I set its objectives, rules, and review criteria, while AI coding agents were the principal implementers, writing, running, testing, and revising the software under my direction. This is what *AI-built* means in this paper. Plectis can expose the resulting public files and runs, but it cannot independently prove that account of how the private work was produced.

The private repository remains private because it contains personal records, credentials, live browser and account information, operator notes, and unfinished tools. Plectis is a separately runnable public collection made from material that could be published or represented with fixed public data. It is not a reconstruction of the private working environment.

The evidence boundary can be stated before any command is run:

Status	What belongs in it
Publicly checkable	The contents of a named public commit, its registered commands, the files those commands write, and whether its public tests pass.
Reported here	The private repository's history, the role of AI agents in producing it, and the private origin of copied material.
Not established here	Whether the public selection represents the private whole, whether the mechanisms generalise, whether their success criteria are independently correct, or whether the private system works reliably as a whole.

2 What the public repository contains

At commit 07ef8b940982, the component registry contains 88 unique entries in 7 groups:

Group	Count	What its components do
Getting started	2	Show a new reader where to begin and guide a first inspection.
Project mapping	12	Map files and written rules, find relevant material, and direct a question to an appropriate part of the project.
Computer-checked mathematics	20	Prepare, repair, or check bounded steps used in formal mathematical work.
AI reliability and safety tests	20	Replay cases involving hostile instructions, software boundaries, stored information, tool use, and benchmark rules.
Research-method tests	9	Exercise fixed cases involving replication, conflicting forecasts, interpretability, simulation, and laboratory safety.
Public-copy maintenance	20	Check copied source, generated pages, publication boundaries, and disagreement between records and derived files.
Work recovery	5	Record unfinished changes and test how work can resume after interruption.

The count is an inventory, not a score. The registry also records how each component is supported. The following four categories cover all 88 entries:

Evidence route	Count	Meaning
Verified copied source	21	The public component carries copied non-secret source bodies and checks them against its public import record.
Public reimplementation	27	The public code reimplements a bounded part of a private mechanism against fixed cases.
Direct or external-tool run	22	The component performs a local computation or invokes a named external tool and records the bounded result.
Contract checker	18	The component recomputes whether a public input satisfies a declared structural or behavioural contract.

Each registry entry names the operation it exercises, the command, its stored receipt paths, the type of evidence, and the limit of the claim. The input path is part of the command. A separate generated page, `ORGANS.md`, gives a plain description and first command for every component.

The validator code still determines what counts as a pass, so a careful review must inspect that criterion rather than infer correctness from the word “pass.”

Every component has a registered command, but not every environment supplies every external tool. A proof-related component, for example, may need Lean. When a required tool is absent, the valid result is a bounded refusal or a locked status, not proof success. Thus *runnable* means that the public operation and its failure boundary are present; it does not mean that all optional dependencies exist on every machine.

3 One component, from input to limit

The component `batch8_audio_level_rms_port` is a small example that does not require knowledge of the private system. It computes a normalised loudness level from synthetic audio sample arrays. It does not open a microphone.

Its public command can write fresh output outside the repository:

```
PYTHONPATH=src python3 -m \
  microcosm_core.organs.batch8_audio_level_rms_port run \
  --input fixtures/first_wave/batch8_audio_level_rms_port/input \
  --out /tmp/plectis-audio-rms \
  --acceptance-out /tmp/plectis-audio-rms-acceptance.json
```

The pass rule is explicit in the validator. Three reference arrays must match their expected values within a stated numerical tolerance. A value above the allowed range must be reduced to one, an empty array must produce zero, and an unknown sample format must be refused. At commit 07ef8b940982, the run produced:

Case	Expected	Observed	Result
Four decimal samples	0.489897949	0.489897949	pass
Four 16-bit integer samples	0.489892970	0.489892970	pass
Samples above the maximum	1.000000000	1.000000000	pass
Empty input	0	0	pass
Unknown sample format	refusal	refusal	pass

The stored reference is the file `batch8_audio_level_rms_port_validation_receipt.json` under `receipts/first_wave/batch8_audio_level_rms_port/`. Its limit is as important as its numbers: this run checks one public Python calculation over synthetic arrays. It does not run the original application, start an audio session, request microphone permission, capture a device, or establish that the chosen expected values are an independent standard.

This example shows what the component format makes inspectable. It does not show that the task is difficult, useful in another project, or representative of the other 87 entries. Since the project supplies both the expected values and the validator, a matching result establishes repeatability under that public test. Independent correctness requires an independent oracle or a fresh input whose expected result was not selected by the maintainer.

4 A clean-clone run

The following source-form command was run in a clean local clone of commit 07ef8b940982 on Darwin 24.6.0 (arm64) with Python 3.13.7:

```
PYTHONPATH=src python3 -m microcosm_core tour --format text .
```

It reported 5,016 project files, found five possible reading routes, selected `readme_onboarding_route`, wrote 21 files under a new `.microcosm/` directory, and reported that the examined source files were unchanged. The output also named three ways to inspect a finding: its evidence, the event record from which it came, and the limit on what it authorises. The complete directory can be deleted and generated again.

Those numbers describe that commit and environment. Installing the package inside the source tree can add packaging metadata, and later commits can add or remove files. Therefore the counts are a dated run result, not a permanent property of Plectis.

For a normal installation, a reader can use:

```
git clone https://github.com/wcook04/plectis
cd plectis
python3 -m pip install .
plectis tour --format text .
```

The core installed package declares no third-party runtime package. The build step does require a sufficiently recent `setuptools`; a cold machine may need to download that build dependency if it is not already installed or cached. After installation, the core component runs are designed to make no network or AI-model calls. Components that wrap external tools have additional environment requirements and must state their limits when those tools are unavailable.

To inspect one claim, a reviewer should read the component's description and limit before running it, run the command on the supplied input, and compare the new result with the stored receipt. A match establishes repeatability. The reviewer should then ask whether the pass rule bears on the claim, perturb the input where practical, and retain failures rather than selecting a more favourable component.

The repository-wide preflight is `make check`. At this snapshot it loads the evidence registry and scans the 58 shipped Lean files for proof placeholders, project-defined axioms, native evaluation, unsafe or partial declarations, and unbounded kernel limits. The broader `make ci` command adds installation, public tests, and two smoke runs. These commands test named properties of the public checkout; they do not certify the private system.

5 Publication records and private provenance

The public repository performs three kinds of consistency check at its publication boundary.

First, copied source bodies have public import records and fingerprints. A fingerprint is a short value computed from file contents; matching values show that the public file agrees with the project's public record. The same maintainer controls both sides of that comparison, so it is internal consistency evidence, not independent proof that the file originated in the private repository.

Second, automated scans check copied material for specified credential, personal-information, and private-path patterns. They also record omitted or rejected material. These checks reduce the chance of accidental disclosure. They do not prove that every possible secret, identifying detail, or derived private fact is absent.

Third, pages such as the component map are generated from structured records that programs can read, and tests compare the generated pages with those records. This detects specified

forms of disagreement and reduces manual drift. A generator or its source record can still be incomplete or wrong, so generation does not make inconsistency impossible.

Plectis is designed not to read the private repository or reconstruct private account state. That design and its scans are public and testable. They reduce risk; they are not an absolute privacy guarantee.

6 What the checks do not establish

A passing run shows that named public code produced a stated result from a supplied input in a stated environment. It does not, without further evidence, establish any of the following:

- that the public components are representative of the private system;
- that the private provenance or AI contribution history is independently verified;
- that a component works on inputs not chosen by the project;
- that a self-authored fixture, expected value, or pass rule is correct;
- that the mechanisms are useful, original, secure, complete, or ready for production;
- that privacy checks exclude every possible disclosure; or
- that the formal-mathematics components prove more than their named public fixtures.

This note does not report an independent, randomly selected audit over fresh inputs. Such an audit would need to freeze a commit, select components without substitution, choose at least some inputs after the freeze, publish raw output and failures, and distinguish repeatability from correctness. That is the clearest next test of the collection.

A separate public repository contains Lean 4 work related to Erdős problems 249 and 257. This paper does not use its module count, declaration count, or association with open problems as evidence for the quality or depth of Plectis. The mathematical repository must be assessed on its own claims and proofs.

What to inspect

The shortest review is to clone the named commit, run the tour, choose one component without first searching for the most favourable example, and inspect its input, validator, output, stored receipt, and limit. `ORGANS.md` maps the components, and `CONTRIBUTING.md` explains how to report a discrepancy. One successful run supports one bounded conclusion. Broader claims require broader evidence.